Restaurant Analytics Using Hadoop and PySpark

1. Introduction

This document explains all the steps performed in the Restaurant Analytics project using Hadoop (HDFS) and PySpark. Each step includes commands, code, and screenshots..

2. Hadoop Setup & Data Upload

Step 1: Start Hadoop

start-dfs.sh
start-yarn.sh

Step 2: Create HDFS Directory and Upload File

hadoop fs -put '/home/cloudera/Desktop/restaurant.csv' /user/cloudera/bigdata_casestudy

```
[cloudera@quickstart ~]$ hadoop fs -put '/home/cloudera/Desktop/restaurant.csv'
/user/cloudera/bigdata_casestudy
```

Step 3: Verify Upload

hadoop fs -ls /user/cloudera/bigdata_casestudy

```
[cloudera@quickstart ~]$ hadoop fs -ls /user/cloudera/bigdata_casestudy
-rw-r--r--   1 cloudera supergroup    2257316 2026-03-21 03:34 /user/cloudera/bi
gdata_casestudy
```

3. Load Data in PySpark

```
>>> from pyspark.sql import SparkSession
>>> from pyspark.sql.functions import *
>>> from pyspark.sql.types import *
>>>
>>> spark = SparkSession.builder \
...     .appName("Restaurant Analytics") \
...     .getOrCreate()
>>>
>>> df = spark.read.csv("/user/cloudera/bigdata_casestudy/", header=True, inferS
chema=True)
2026-03-21 03:49:29 WARN  NativeCodeLoader:62 - Unable to load native-hadoop lib
rary for your platform... using builtin-java classes where applicable
>>> df.show(5)
```

4. Data Cleaning Pipeline

```
>>> df=df.dropna()
>>>
```

5. Use Case 1: Global Cuisine Popularity Analysis

df_cuisine = df.withColumn("Cuisine", explode(split(col("Cuisines"), ", ")))

# Global popularity
global_cuisine = df_cuisine.groupBy("Cuisine").count().orderBy(desc("count"))

# City-wise popularity
city_cuisine = df_cuisine.groupBy("City", "Cuisine").count() \
.orderBy("City", desc("count"))

```
>>> df_cuisine = df.withColumn("Cuisine", explode(split(col("Cuisines"), ", ")))
>>>
>>>
>>> global_cuisine = df_cuisine.groupBy("Cuisine").count().orderBy(desc("count"))
>>>
>>>
>>> city_cuisine = df_cuisine.groupBy("City", "Cuisine").count() \
...                          .orderBy("City", desc("count"))
```

```
>>> global_cuisine.show(3)
+------------+-----+
|     Cuisine|count|
+------------+-----+
|North Indian| 3958|
|     Chinese| 2733|
|   Fast Food| 1984|
+------------+-----+
only showing top 3 rows

>>> city_cuisine.show(5)
+---------+------------+-----+
|     City|     Cuisine|count|
+---------+------------+-----+
|Abu Dhabi|      Indian|    7|
|Abu Dhabi|    American|    4|
|Abu Dhabi|     Chinese|    3|
|Abu Dhabi|North Indian|    3|
|Abu Dhabi|    Japanese|    2|
```

6. Use Case 2: Normalizing and Comparing Costs Across Currencies

Currency column having currency code

```
>>> df.select("Currency").distinct().show(50, False)
+----------------------+
|Currency              |
+----------------------+
|Qatari Rial(QR)       |
|Sri Lankan Rupee(LKR) |
|NewZealand($)         |
|Emirati Diram(AED)    |
|Turkish Lira(TL)      |
|Indian Rupees(Rs.)    |
|Dollar($)             |
|Pounds(??)            |
|Rand(R)               |
|Indonesian Rupiah(IDR)|
|Botswana Pula(P)      |
|Brazilian Real(R$)    |
+----------------------+
```

Adding new column currency code by extracting codes and creating new df for currency code and conversion rate

```
>>> df = df.withColumn(
...     "Currency_Code",
...     regexp_extract(col("Currency"), r'\((.*?)\)', 1)
... )
>>>
>>> rates = spark.createDataFrame([
...     ("$", 1.0),
...     ("Rs.", 0.012),
...     ("£", 1.25),
...     ("QR", 0.27),
...     ("AED", 0.27),
...     ("R", 0.055),
...     ("P", 0.075),
...     ("NZ$", 0.60),
...     ("R$", 0.20),
...     ("IDR", 0.000065),
...     ("LKR", 0.0031),
...     ("TL", 0.03)
... ], ["Currency_Code", "Rat cloudera@quickstart:~
```

```
>>> df_cost = df.join(broadcast(rates), "Currency_Code", "inner")
>>>
>>> df_cost = df_cost.withColumn(
...     "Cost_USD",
...     col("Average Cost for two") * col("Rate")
... )
>>>
>>> city_cost = df_cost.groupBy("City") \
...     .agg(avg("Cost_USD").alias("Avg_Cost_USD")) \
...     .orderBy(desc("Avg_Cost_USD"))
>>>
```

```
>>> city_cost.show(10, False)
+----------------+------------------+
|City            |Avg_Cost_USD      |
+----------------+------------------+
|Pasay City      |250.0             |
|Singapore       |158.94736842105263|
|Mandaluyong City|150.0             |
|Taguig City     |129.375           |
|Paynesville     |120.0             |
|Tagaytay City   |90.0              |
|Makati City     |86.25             |
|Wellington City |71.25             |
|Princeton       |70.0              |
|Pasig City      |70.0              |
+----------------+------------------+
only showing top 10 rows
```

cloudera@quickstart

7. Use Case 3:Impact of Services on Customer Ratings

Restaurants offering online delivery and table booking tend to have higher average ratings, indicating better customer satisfaction

```
>>> online_delivery = df.groupBy("Has Online delivery") \
...     .agg(
...         avg("Aggregate rating").alias("Avg_Rating"),
...         count("*").alias("Total_Restaurants")
...     )
>>>
>>> table_booking = df.groupBy("Has Table booking") \
...     .agg(
...         avg("Aggregate rating").alias("Avg_Rating"),
...         count("*").alias("Total_Restaurants")
...     )
```

```
>>> online_delivery.show()
+-------------------+------------------+-----------------+
|Has Online delivery|        Avg_Rating|Total_Restaurants|
+-------------------+------------------+-----------------+
|                 No|2.4638441998306515|             7086|
|                Yes|3.2486938775510255|             2450|
+-------------------+------------------+-----------------+

>>> table_booking.show()
+-----------------+------------------+-----------------+
|Has Table booking|        Avg_Rating|Total_Restaurants|
+-----------------+------------------+-----------------+
|               No|  2.55816423967534|             8378|
|              Yes|3.4419689119170966|             1158|
+-----------------+------------------+-----------------+
```

8. Use Case 4: Identifying "Hidden Gems" (Weighted Rating Analysis)

The weighted rating method is used to provide a more reliable ranking of restaurants by considering both the average rating and the number of votes received.

The formula used is:

WR= [v /(v+m)] * R + [m /(v+m)]*c

Where:

- WR = Weighted Rating
- R = Average rating of the restaurant
- v = Number of votes received by the restaurant
- m = Minimum votes required (threshold value)
- C = Overall average rating of all restaurants

```
>>> C = df.select(avg("Aggregate rating")).first()[0]
>>> m = 100
>>>
>>> df_hidden = df.withColumn(
...     "Weighted_Rating",
...     (col("Votes")/(col("Votes")+m))*col("Aggregate rating") +
...     (m/(col("Votes")+m))*lit(C)
... )
>>>
>>> df_hidden.select("Restaurant Name", "City", "Aggregate rating", "Votes", "Weighted_Rating") \
...     .orderBy(desc("Weighted_Rating")) \
...     .show(10, False)
```

```
+----------------------------+---------+----------------+-----+------------------+
|Restaurant Name             |City     |Aggregate rating|Votes|Weighted_Rating   |
+----------------------------+---------+----------------+-----+------------------+
|Barbeque Nation             |Kolkata  |4.9             |5966 |4.863163347749698 |
|AB's - Absolute Barbecues   |Hyderabad|4.9             |5434 |4.85962213000536  |
|Mirchi And Mime             |Mumbai   |4.9             |3244 |4.833178489069876 |
|Naturals Ice Cream          |New Delhi|4.9             |2620 |4.817848848327083 |
|McGuire's Irish Pub & Brewery|Pensacola|4.9             |2238 |4.804426376154691 |
|Talaga Sampireun            |Tangerang|4.9             |2212 |4.803351586267156 |
|Indian Accent - The Manor   |New Delhi|4.9             |1934 |4.790142019395115 |
|Toit                        |Bangalore|4.8             |10934|4.780655144775209 |
|Barbeque Nation             |Kolkata  |4.9             |1753 |4.7794111535076444|
|Talaga Sampireun            |Jakarta  |4.9             |1662 |4.773183239188232 |
+----------------------------+---------+----------------+-----+------------------+
only showing top 10 rows
```

9. Use Case 5: Restaurant Clustering for Marketing

Restaurants were segmented into Budget, Mid-range, and Luxury categories using rule-based clustering based on cost

```
>>> df_cluster = df.withColumn(
...     "Cluster",
...     when(col("Average Cost for two") <= 500, "Budget")
...     .when((col("Average Cost for two") > 500) & (col("Average Cost for two")
 <= 2000), "Mid-Range")
...     .otherwise("Luxury")
... )
>>>
>>> df_cluster.select("Restaurant Name", "City", "Average Cost for two", "Cluste
r") \
...     .show(20, False)
```

```
---------+
|Restaurant Name                        |City           |Average Cost for two|
Cluster  |
+---------------------------------------+---------------+-------------------+
---------+
|Le Petit Souffle                       |Makati City    |1100               |
Mid-Range|
|Izakaya Kikufuji                       |Makati City    |1200               |
Mid-Range|
|Heat - Edsa Shangri-La                 |Mandaluyong City|4000              |
Luxury   |
|Ooma                                   |Mandaluyong City|1500              |
Mid-Range|
|Sambo Kojin                            |Mandaluyong City|1500              |
Mid-Range|
|Din Tai Fung                           |Mandaluyong City|1000              |
Mid-Range|
|Buffet 101                             |Pasay City     |2000               |
```

```
>>> df_cluster.groupBy("Cluster").count().show()
+---------+-----+
|  Cluster|count|
+---------+-----+
|   Budget| 6070|
|   Luxury|  270|
|Mid-Range| 3217|
+---------+-----+
```

KMeans clustering produces variable group sizes because it is data-driven, unlike rule-based classification in PySpark.

```
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

X = df[["Average_Cost_for_two", "Votes", "Aggregate_rating"]]

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

kmeans = KMeans(n_clusters=3, random_state=42)
df["Cluster"] = kmeans.fit_predict(X_scaled)

print(df["Cluster"].value_counts())
```

[23] ✓ 0.0s

```
Cluster
0    7161
1    2148
2     242
Name: count, dtype: int64
```

10. Use Case 6: Geolocation-based Food Hub Detection

Nearby restaurants were grouped by rounding latitude and longitude. Locations with high counts were identified as food hubs.To improve interpretability, city and locality information was added to geolocation clusters, making it easier to identify real-world food hubs

```
>>> from pyspark.sql.functions import round, count, desc, first
>>>
>>> df_geo = df.withColumn("Lat_Round", round(col("Latitude"), 2)) \
...            .withColumn("Lon_Round", round(col("Longitude"), 2))
>>>
>>> food_hubs = df_geo.groupBy("Lat_Round", "Lon_Round") \
...     .agg(
...         count("*").alias("Restaurant_Count"),
...         first("City").alias("City"),
...         first("Locality").alias("Locality"),
...         first("Restaurant Name").alias("Sample_Restaurant")
...     ) \
...     .orderBy(desc("Restaurant_Count"))
>>>
>>> food_hubs.select("City", "Locality", "Restaurant_Count", "Sample_Restaurant") \
...     .show(10, False)

+---------+-------------------------------+----------------+----------------------------+
|City     |Locality                       |Restaurant_Count|Sample_Restaurant           |
+---------+-------------------------------+----------------+----------------------------+
|Agra     |Agra Cantt                     |497             |Rangrezz Restaurant         |
|New Delhi|Barakhamba Road                |147             |Makers & Bakers             |
|Noida    |Centre Stage Mall, Sector 18   |104             |Geoffrey's                  |
|Noida    |Brahmaputra Shopping Complex   |79              |JSB Evergreen Snack & Sweets|
|New Delhi|Satyaniketan                   |76              |Alfa Kathi Rolls            |
|New Delhi|Green Park                     |72              |Cafe Coffee Day             |
|New Delhi|East Patel Nagar               |72              |Naivedyam                   |
|New Delhi|City Square Mall, Rajouri Garden|71              |Cafe Cruise                 |
|Gurgaon  |Cyber Greens, DLF Cyber City   |70              |Isabella                    |
|New Delhi|East of Kailash                |64              |Kowloon Express             |
+---------+-------------------------------+----------------+----------------------------+
only showing top 10 rows
```

11. Use Case 7: Data Quality & Standardization Pipeline

```
>>> from pyspark.sql.functions import col, lower, trim
>>>
>>> df_clean = df.withColumn("City", lower(trim(col("City")))) \
...               .withColumn("Locality", lower(trim(col("Locality")))) \
...               .withColumn("Cuisines", trim(col("Cuisines")))
```

Since null values were already removed in earlier steps, this stage focused on standardizing text fields such as city, locality, and cuisines by trimming spaces and converting to lowercase for consistency

12. Use Case 8: Locality-wise Price Range Distribution

The distribution of restaurant price ranges was analyzed for each locality. Percentage calculations helped identify whether an area is dominated by budget or premium restaurants.

```
>>> from pyspark.sql.functions import count, col
>>>
>>> price_dist = df_clean.groupBy("Locality", "Price range") \
...     .agg(count("*").alias("Count"))
>>>
>>> total = df_clean.groupBy("Locality") \
...     .agg(count("*").alias("Total"))
>>>
>>> result = price_dist.join(total, "Locality") \
...     .withColumn("Percentage", (col("Count")/col("Total"))*100)
>>>
```

```
>>> result.select("Locality", "Price range", "Percentage") \
...     .show(20, False)
+-----------------------------------+-----------+------------------+
|Locality                           |Price range|Percentage        |
+-----------------------------------+-----------+------------------+
|satwa                              |3          |100.0             |
|indiranagar                        |3          |33.33333333333333 |
|surya nagar                        |1          |50.0              |
|lajpat nagar 4                     |1          |50.0              |
|model town 2                       |3          |8.0               |
|moti nagar                         |2          |40.0              |
|the lalit new delhi, barakhamba road|3          |12.5              |
|spice world mall, sector 25        |2          |50.0              |
|lalpur                             |3          |16.666666666666664|
|te aro                             |2          |13.333333333333334|
```

13. Use Case 9: Price Range Prediction

The objective of this use case is to predict the price range category (1 to 4) of a restaurant based on its average cost for two people. A rule-based approach was implemented using predefined cost thresholds. Restaurants were categorized into different price ranges such as budget, mid-range, and luxury. This helps in understanding pricing patterns and supports decision-making for new restaurant setups

Price Range	Meaning
1	Cheap / Budget
2	Moderate / Affordable
3	Expensive
4	Very Expensive / Luxury

```
>>> from pyspark.sql.functions import when, col
>>>
>>> df_pred = df_clean.withColumn(
...     "Predicted_Price_Range",
...     when(col("Average_Cost_for_two") <= 500, 1)
...     .when((col("Average_Cost_for_two") > 500) & (col("Average_Cost_for_two") <= 1500), 2)
...     .when((col("Average_Cost_for_two") > 1500) & (col("Average_Cost_for_two") <= 3000), 3)
...     .otherwise(4)
... )
>>>
>>> df_pred.select(
...     "Restaurant_Name",
...     "City",
...     "Average_Cost_for_two",
...     "Price_range",
...     "Predicted_Price_Range"
... ).show(20, False)
```

```
+-------------------------------------+---------------+-------------------+-----------+--------------------+
|Restaurant_Name                      |City           |Average_Cost_for_two|Price_range|Predicted_Price_Range|
+-------------------------------------+---------------+-------------------+-----------+--------------------+
|Le Petit Souffle                     |Makati City    |1100               |3          |2                   |
|Izakaya Kikufuji                     |Makati City    |1200               |3          |2                   |
|Heat - Edsa Shangri-La               |Mandaluyong City|4000              |4          |4                   |
|Ooma                                 |Mandaluyong City|1500              |4          |2                   |
|Sambo Kojin                          |Mandaluyong City|1500              |4          |2                   |
|Din Tai Fung                         |Mandaluyong City|1000              |3          |2                   |
|Buffet 101                           |Pasay City     |2000               |4          |3                   |
|Vikings                              |Pasay City     |2000               |4          |3                   |
|Spiral - Sofitel Philippine Plaza Manila|Pasay City     |6000              |4          |4                   |
|Locavore                             |Pasig City     |1100               |3          |2                   |
|Silantro Fil-Mex                     |Pasig City     |800                |3          |2                   |
|Mad Mark's Creamery & Good Eats      |Pasig City     |900                |3          |2                   |
|Silantro Fil-Mex                     |Quezon City    |800                |3          |2                   |
|Guevarra's                           |San Juan City  |1000               |3          |2                   |
|Sodam Korean Restaurant              |San Juan City  |700                |3          |2                   |
```

14. Use Case 10: Top-N Recommendation Engine

Recommending similar restaurants to a user,The recommendation system filters restaurants based on the same city as the selected restaurant to ensure location relevance. The condition Restaurant_Name != "Vikings" is used to exclude the selected restaurant itself from the results, so that only other similar restaurants are recommended.

```
>>> from pyspark.sql.functions import col, desc, split, explode
>>>
>>> recommend = df_clean.filter(
...     (col("City") == fav_city) &
...     (col("Restaurant_Name") != "Vikings")
... )
>>>
>>> recommend = recommend.withColumn(
...     "Cuisine",
...     explode(split(col("Cuisines"), ","))
... )
>>>
>>> top5 = recommend.orderBy(desc("Aggregate_rating")) \
...     .select("Restaurant_Name", "City", "Cuisine", "Aggregate_rating") \
...     .limit(5)
```

```
>>> top5.show(5, False)
+---------------------------------------+----------+---------+----------------+
|Restaurant_Name                        |City      |Cuisine  |Aggregate_rating|
+---------------------------------------+----------+---------+----------------+
|Spiral - Sofitel Philippine Plaza Manila|Pasay City| Indian  |4.9             |
|Spiral - Sofitel Philippine Plaza Manila|Pasay City| Asian   |4.9             |
|Spiral - Sofitel Philippine Plaza Manila|Pasay City|European |4.9             |
|Buffet 101                             |Pasay City|Asian    |4               |
|Buffet 101                             |Pasay City| European|4               |
+---------------------------------------+----------+---------+----------------+
```

15. Use Case 11: Optimized Currency Conversion (Broadcast Join)

A broadcast join is a performance optimization in distributed computing that joins a small table with a large table by copying the small table to every worker node. It eliminates costly network shuffling of the large table, dramatically speeding up joins and reducing memory overhead

```
>>> df_cost = df.join(broadcast(rates), "Currency_Code", "inner")
>>>
```

```
>>> df_cost.select(
...     "Restaurant Name",
...     "City",
...     "Currency_Code",
...     "Average Cost for two"
... ).show(10, False)
+---------------------------------------+----------------+-------------+--------------------+
|Restaurant Name                        |City            |Currency_Code|Average Cost for two|
+---------------------------------------+----------------+-------------+--------------------+
|Le Petit Souffle                       |Makati City     |P            |1100                |
|Izakaya Kikufuji                       |Makati City     |P            |1200                |
|Heat - Edsa Shangri-La                 |Mandaluyong City|P            |4000                |
|Ooma                                   |Mandaluyong City|P            |1500                |
|Sambo Kojin                            |Mandaluyong City|P            |1500                |
|Din Tai Fung                           |Mandaluyong City|P            |1000                |
|Buffet 101                             |Pasay City      |P            |2000                |
|Vikings                                |Pasay City      |P            |2000                |
|Spiral - Sofitel Philippine Plaza Manila|Pasay City      |P            |6000                |
|Locavore                               |Pasig City      |P            |1100                |
```

16. Use Case 12: Efficient Data Pipelines (Caching & Storage)

Cache used to store intermediate result to avoid recomputing and improve performance

```
>>> df_clean = df.dropna()
>>>
>>> df_clean.cache()
DataFrame[Restaurant ID: string, Restaurant Name: string, Country Code: string, City: string, Address: string, Locality: string, Loc
ality Verbose: string, Longitude: string, Latitude: string, Cuisines: string, Average Cost for two: string, Currency: string, Has Ta
ble booking: string, Has Online delivery: string, Is delivering now: string, Switch to order menu: string, Price range: string, Aggr
egate rating: string, Rating color: string, Rating text: string, Votes: int, Currency_Code: string]
>>>
>>> df_clean.count()
9536
```

```
>>> df_clean = df_clean.toDF(*[c.replace(" ", "_") for c in df_clean.columns])
>>> df_clean.write.mode("overwrite").parquet("/user/cloudera/cleaned_data/")
```

```
[cloudera@quickstart ~]$ hadoop fs -ls /user/cloudera/cleaned_data/
Found 2 items
-rw-r--r--   1 cloudera supergroup          0 2026-03-21 06:13 /user/cloudera/cl
eaned_data/_SUCCESS
-rw-r--r--   1 cloudera supergroup     622034 2026-03-21 06:13 /user/cloudera/cl
eaned_data/part-00000-9c526ff1-3747-48b3-8b8f-c62a93f52c07-c000.snappy.parquet
[cloudera@quickstart ~]$
```

The cleaned dataset was successfully stored in Parquet format in HDFS after standardizing column names. This ensures efficient storage and faster query performance

17. Use Case 13: Service Impact Comparative Visualization

The objective of this use case is to analyze how additional services such as online delivery and table booking influence customer ratings. The dataset was processed in PySpark to aggregate relevant features like ratings, votes, and price range. This analysis helps in understanding whether such services contribute to improved customer satisfaction.

```
>>> from pyspark.sql.functions import avg, count
>>>
>>> df_clean.groupBy("Has_Online_delivery") \
...     .agg(
...         avg("Aggregate_rating").alias("Avg_Rating"),
...         count("*").alias("Total_Restaurants")
...     ) \
...     .show()
+-------------------+------------------+-----------------+
|Has_Online_delivery|        Avg_Rating|Total_Restaurants|
+-------------------+------------------+-----------------+
|                 No|2.4638441998306515|             7086|
|                Yes|3.2486938775510255|             2450|
+-------------------+------------------+-----------------+
```

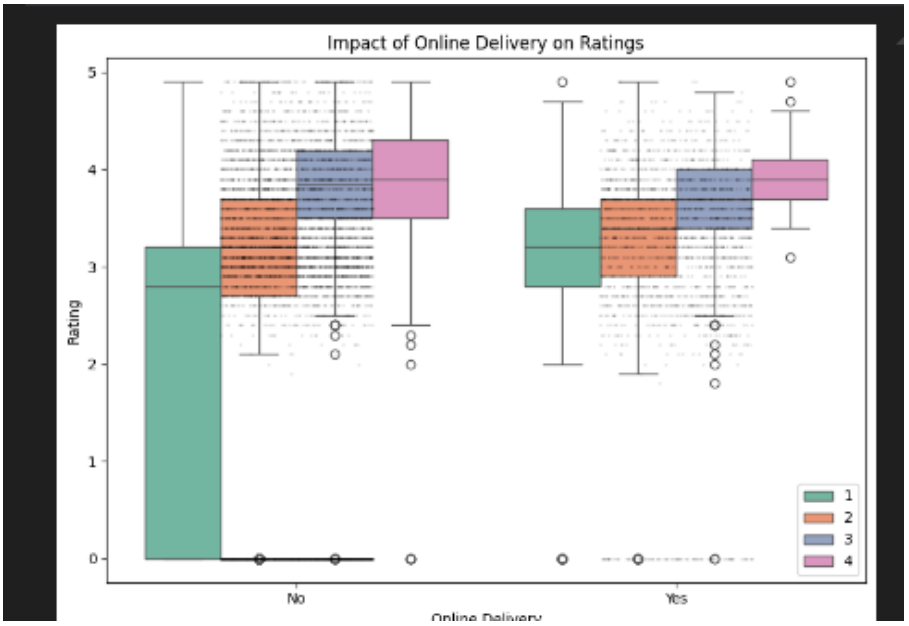
```
>>>
>>> df_clean.groupBy("Has_Table_booking") \
...     .agg(
...         avg("Aggregate_rating").alias("Avg_Rating"),
...         count("*").alias("Total_Restaurants")
...     ) \
...     .show()

+-----------------+------------------+-----------------+
|Has_Table_booking|        Avg_Rating|Total_Restaurants|
+-----------------+------------------+-----------------+
|               No|  2.55816423967534|             8378|
|              Yes|3.4419689119170966|             1158|
+-----------------+------------------+-----------------+
```

```
>>> df_clean.groupBy("Has_Online_delivery", "Price_range") \
...     .agg(
...         avg("Aggregate_rating").alias("Avg_Rating")
...     ) \
...     .show()
+-------------------+-----------+------------------+
|Has_Online_delivery|Price_range|        Avg_Rating|
+-------------------+-----------+------------------+
|               Yes|          3|3.6277372262773713|
|               Yes|          2| 3.226848249027234|
|                No|          1| 1.807093147751601|
|                No|          3|3.7053319919517107|
|                No|          2|2.7404605263157893|
|               Yes|          4|3.9301886792452843|
|               Yes|          1| 3.014978601997144|
|                No|          4|3.8078947368421066|
+-------------------+-----------+------------------+
```

cloudera@quickstart:~

A box plot was used to compare rating distributions across online delivery services and price ranges. Jitter points were initially added to show data density but removed in the final visualization to improve clarity.i saved the data from hadoop to local and visualised using jupyter labs


Impact of Online Delivery on Ratings
Rating
5
4
3
2
1
0
No
Yes
Online Delivery
1
2
3
4